Description | Note | Editorial | Solutions | Submissions

3512. Minimum Operations to Make Array Sum Divisible by K

Solved

Easy | Topics | Companies | Hint

You are given an integer array `nums` and an integer `k`. You can perform the following operation any number of times:

- Select an index `i` and replace `nums[i]` with `nums[i] - 1`.

Return the **minimum** number of operations required to make the sum of the array divisible by `k`.

Example 1:

> **Input:** `nums = [3,9,7], k = 5`
>
> **Output:** 4
>
> **Explanation:**
>
> - Perform 4 operations on `nums[1] = 9`. Now, `nums = [3, 5, 7]`.
> - The sum is 15, which is divisible by 5.

Example 2:

> **Input:** `nums = [4,1,3], k = 4`
>
> **Output:** `0`
>
> **Explanation:**

352 | 229 | 1 Online

Code

Java | Auto

```java
class Solution {
    public int minOperations(int[] nums, int k) {
        long totalSum = 0;
        for (int num : nums) {
            totalSum += num;
        }
        return (int) (totalSum % k);
    }

    public static void main(String[] args) {
        Solution solver = new Solution();
        int[] nums = {3, 9, 7};
        int k = 5;
        System.out.println("Minimum operations: " + solver.minOperations(nums, k));
    }
}
```

Testcase | Test Result

Case 1 | Case 2 | Case 3 | +

nums =

[3,9,7]

k =

5

Source

Description | Note | Editorial | Solutions | Submissions

3925. Concatenate Array With Reverse

Solved

Easy | Topics | Companies | Hint

You are given an integer array `nums` of length `n`.

Construct a new array `ans` of length `2 * n` such that the first `n` elements are the same as `nums`, and the next `n` elements are the elements of `nums` in reverse order.

Formally, for `0 <= i <= n - 1`:

- `ans[i] = nums[i]`
- `ans[i + n] = nums[n - i - 1]`

Return an integer array `ans`.

Example 1:

> **Input:** `nums = [1,2,3]`
>
> **Output:** `[1,2,3,3,2,1]`
>
> **Explanation:**
>
> The first `n` elements of `ans` are the same as `nums`.
>
> For the next `n = 3` elements, each element is taken from `nums` in reverse order:
>
> - `ans[3] = nums[2] = 3`
> - `ans[4] = nums[1] = 2`

31 | 15 | 6 Online

Code

Java | Auto

```java
class Solution {
    public int[] concatWithReverse(int[] nums) {
        int n = nums.length;
        int[] ans = new int[2 * n];

        for (int i = 0; i < n; i++) {
            ans[i] = nums[i];
            ans[i + n] = nums[n - 1 - i];
        }

        return ans;
    }
}
```

Saved | Ln 13, Col 2

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2

Input

nums =
`[1,2,3]`

Output

`[1,2,3,3,2,1]`

Expected

Description | Note | Editorial | Solutions | Submissions

3668. Restore Finishing Order

Solved

Easy | Topics | Companies | Hint

You are given an integer array `order` of length `n` and an integer array `friends`.

- `order` contains every integer from 1 to `n` **exactly once**, representing the IDs of the participants of a race in their **finishing** order.
- `friends` contains the IDs of your friends in the race **sorted** in strictly increasing order. Each ID in friends is guaranteed to appear in the `order` array.

Return an array containing your friends' IDs in their **finishing** order.

Example 1:

> **Input:** `order = [3,1,2,5,4], friends = [1,3,4]`
>
> **Output:** `[3,1,4]`
>
> **Explanation:**
>
> The finishing order is `[3, 1, 2, 5, 4]`. Therefore, the finishing order of your friends is `[3, 1, 4]`.

Example 2:

> **Input:** `order = [1,4,5,3,2], friends = [2,5]`
>
> **Output:** `[5,2]`
>
> **Explanation:**

131 | 30 | 9 Online

Code

Java | Auto

```java
class Solution {
    public int[] recoverOrder(int[] order, int[] friends) {
        HashSet<Integer> set = new HashSet<>();

        for (int f : friends) {
            set.add(f);
        }

        int[] ans = new int[friends.length];
        int index = 0;

        for (int x : order) {
            if (set.contains(x)) {
                ans[index++] = x;
            }
        }

        return ans;
    }
}
```

Testcase | Test Result

Case 1 | Case 2

order =

```
[3,1,2,5,4]
```

friends =

```
[1,3,4]
```

Source

Description | Note | Editorial | Solutions | Submissions

1920. Build Array from Permutation

Solved

Easy | Topics | Companies | Hint

Given a **zero-based permutation** `nums` (**0-indexed**), build an array `ans` of the **same length** where `ans[i] = nums[nums[i]]` for each `0 <= i < nums.length` and return it.

A **zero-based permutation** `nums` is an array of **distinct** integers from `0` to `nums.length - 1` (**inclusive**).

Example 1:

```
Input: nums = [0,2,1,5,3,4]
Output: [0,1,2,4,5,3]
Explanation: The array ans is built as follows:
ans = [nums[nums[0]], nums[nums[1]], nums[nums[2]],
nums[nums[3]], nums[nums[4]], nums[nums[5]]]
    = [nums[0], nums[2], nums[1], nums[5], nums[3], nums[4]]
    = [0,1,2,4,5,3]
```

Example 2:

```
Input: nums = [5,0,1,2,3,4]
Output: [4,5,0,1,2,3]
Explanation: The array ans is built as follows:
ans = [nums[nums[0]], nums[nums[1]], nums[nums[2]],
nums[nums[3]], nums[nums[4]], nums[nums[5]]]
    = [nums[5], nums[0], nums[1], nums[2], nums[3], nums[4]]
    = [4,5,0,1,2,3]
```

4.1K | 423 | 9 Online

Code

Java | Auto

```java
class Solution {
    public int[] buildArray(int[] nums) {
        int[] ans = new int[nums.length];

        for (int i = 0; i < nums.length; i++) {
            ans[i] = nums[nums[i]];
        }

        return ans;
    }
}
```

Saved | Ln 1, Col 1

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2

Input

nums =

[0,2,1,5,3,4]

Output

[0,1,2,4,5,3]

Expected

Description | Note | Editorial | Solutions | Submissions

3190. Find Minimum Operations to Make All Elements Divisible by Three

Solved

Easy | Topics | Companies | Hint

You are given an integer array `nums`. In one operation, you can add or subtract 1 from **any** element of `nums`.

Return the **minimum** number of operations to make all elements of `nums` divisible by 3.

Example 1:

> **Input:** `nums = [1,2,3,4]`
>
> **Output:** 3
>
> **Explanation:**
>
> All array elements can be made divisible by 3 using 3 operations:
>
> - Subtract 1 from 1.
> - Add 1 to 2.
> - Subtract 1 from 4.

Example 2:

> **Input:** `nums = [3,6,9]`
>
> **Output:** `0`

531 | 198 | 4 Online

Code

Java | Auto

```java
class Solution {
    public int minimumOperations(int[] nums) {
        int count = 0;

        for (int num : nums) {
            if (num % 3 != 0) {
                count++;
            }
        }

        return count;
    }
}
```

Saved | Ln 1, Col 1

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2

Input

nums =
[1,2,3,4]

Output

3

Expected

Description | Note | Editorial | Solutions | Submissions

3838. Weighted Word Mapping

Solved

Easy | Topics | Companies | Hint

You are given an array of strings `words`, where each string represents a word containing lowercase English letters.

You are also given an integer array `weights` of length 26, where `weights[i]` represents the weight of the `i^th` lowercase English letter.

The **weight** of a word is defined as the **sum** of the weights of its characters.

For each word, take its weight modulo 26 and map the result to a lowercase English letter using reverse alphabetical order (`0 -> 'z', 1 -> 'y', ..., 25 -> 'a'`).

Return a string formed by concatenating the mapped characters for all words in order.

Example 1:

> **Input:** `words = ["abcd","def","xyz"], weights = [5,3,12,14,1,2,3,2,10,6,6,9,7,8,7,10,8,9,6,9,9,8,3,7,7,2]`
>
> **Output:** `"rij"`
>
> **Explanation:**
>
> - The weight of `"abcd"` is `5 + 3 + 12 + 14 = 34`. The result modulo 26 is `34 % 26 = 8`, which maps to `'r'`.
> - The weight of `"def"` is `14 + 1 + 2 = 17`. The result modulo 26 is `17 % 26 = 17`, which maps to `'i'`.

233 | 89 | 15 Online

Code

Java | Auto

```java
class Solution {
    public String mapWordWeights(String[] words, int[] weights) {
        StringBuilder ans = new StringBuilder();

        for (String word : words) {
            int sum = 0;

            for (char ch : word.toCharArray()) {
                sum += weights[ch - 'a'];
            }

            ans.append((char)('z' - (sum % 26)));
        }

        return ans.toString();
    }
}
```

Saved | Ln 16, Col 6

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

words =

`["abcd","def","xyz"]`

weights =

`[5,3,12,14,1,2,3,2,10,6,6,9,7,8,7,10,8,9,6,9,9,8,3,7,7,2]`

Description | Note | Editorial | Solutions | Submissions

2011. Final Value of Variable After Performing Operations

Solved

Easy | Topics | Companies | Hint

There is a programming language with only **four** operations and **one** variable `X`:

- `++X` and `X++` **increments** the value of the variable `X` by `1`.
- `--X` and `X--` **decrements** the value of the variable `X` by `1`.

Initially, the value of `X` is `0`.

Given an array of strings `operations` containing a list of operations, return *the **final** value of `X` after performing all the operations*.

Example 1:

```
Input: operations = ["--X","X++","X++"]
Output: 1
Explanation: The operations are performed as follows:
Initially, X = 0.
--X: X is decremented by 1, X =  0 - 1 = -1.
X++: X is incremented by 1, X = -1 + 1 =  0.
X++: X is incremented by 1, X =  0 + 1 =  1.
```

Example 2:

```
Input: operations = ["++X","++X","X++"]
Output: 3
Explanation: The operations are performed as follows:
Initially, X = 0.
++X: X is incremented by 1, X = 0 + 1 = 1.
```

2K | 312 | 8 Online

Code

Java | Auto

```java
class Solution {
    public int finalValueAfterOperations(String[] operations) {
        int x = 0;
        for (String op : operations) {
            if (op.charAt(1) == '+') {
                x++;
            } else {
                x--;
            }
        }
        return x;
    }
}
```

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

operations =

["--X","X++","X++"]

Output

1

Expected

1

Description | Note | Editorial | Solutions | Submissions

2942. Find Words Containing Character

Solved

Easy | Topics | Companies | Hint

You are given a **0-indexed** array of strings `words` and a character `x`.

Return *an **array of indices** representing the words that contain the character* `x`.

Note that the returned array may be in **any** order.

Example 1:

```
Input: words = ["leet","code"], x = "e"
Output: [0,1]
Explanation: "e" occurs in both words: "leet", and "code".
Hence, we return indices 0 and 1.
```

Example 2:

```
Input: words = ["abc","bcd","aaaa","cbc"], x = "a"
Output: [0,2]
Explanation: "a" occurs in "abc", and "aaaa". Hence, we
return indices 0 and 2.
```

Example 3:

```
Input: words = ["abc","bcd","aaaa","cbc"], x = "z"
Output: []
Explanation: "z" does not occur in any of the words. Hence,
we return an empty array.
```

Constraints:

717 | 123 | 8 Online

Code

Java | Auto

```java
class Solution {
    public List<Integer> findWordsContaining(String[] words, char x) {
        List<Integer> ans = new ArrayList<>();

        for (int i = 0; i < words.length; i++) {
            if (words[i].indexOf(x) != -1) {
                ans.add(i);
            }
        }

        return ans;
    }
}
```

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

words =
["leet","code"]

x =
"e"

Output

[0,1]

Description | Note | Editorial | Solutions | Submissions

1929. Concatenation of Array

Solved

Easy | Topics | Companies | Hint

Given an integer array `nums` of length `n`, you want to create an array `ans` of length `2n` where `ans[i] == nums[i]` and `ans[i + n] == nums[i]` for `0 <= i < n` (**0-indexed**).

Specifically, `ans` is the **concatenation** of two `nums` arrays.

Return *the array* `ans`.

Example 1:

```
Input: nums = [1,2,1]
Output: [1,2,1,1,2,1]
Explanation: The array ans is formed as follows:
- ans = [nums[0],nums[1],nums[2],nums[0],nums[1],nums[2]]
- ans = [1,2,1,1,2,1]
```

Example 2:

```
Input: nums = [1,3,2,1]
Output: [1,3,2,1,1,3,2,1]
Explanation: The array ans is formed as follows:
- ans =
[nums[0],nums[1],nums[2],nums[3],nums[0],nums[1],nums[2],nums[3]]
- ans = [1,3,2,1,1,3,2,1]
```

Constraints:

4.3K | 238 | 76 Online

Code

Java | Auto

```java
class Solution {
    public int[] getConcatenation(int[] nums) {
        int n = nums.length;
        int[] ans = new int[2 * n];

        for (int i = 0; i < n; i++) {
            ans[i] = nums[i];
            ans[i + n] = nums[i];
        }

        return ans;
    }
}
```

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2

Input

nums =
[1,2,1]

Output

[1,2,1,1,2,1]

Expected

[1,2,1,1,2,1]

Description | Note | Editorial | Solutions | Submissions

3701. Compute Alternating Sum

Solved

Easy | Topics | Companies | Hint

You are given an integer array `nums`.

The **alternating sum** of `nums` is the value obtained by **adding** elements at even indices and **subtracting** elements at odd indices. That is, `nums[0] - nums[1] + nums[2] - nums[3]...`

Return an integer denoting the alternating sum of `nums`.

Example 1:

> **Input:** `nums = [1,3,5,7]`
>
> **Output:** `-4`
>
> **Explanation:**
>
> - Elements at even indices are `nums[0] = 1` and `nums[2] = 5` because 0 and 2 are even numbers.
> - Elements at odd indices are `nums[1] = 3` and `nums[3] = 7` because 1 and 3 are odd numbers.
> - The alternating sum is `nums[0] - nums[1] + nums[2] - nums[3] = 1 - 3 + 5 - 7 = -4`.

Example 2:

> **Input:** `nums = [100]`
>
> **Output:** `100`

74 | 33 | 6 Online

Code

Java | Auto

```java
class Solution {
    public int alternatingSum(int[] nums) {
        int sum = 0;

        for (int i = 0; i < nums.length; i++) {
            if (i % 2 == 0) {
                sum += nums[i];
            } else {
                sum -= nums[i];
            }
        }
```

Saved | Ln 15, Col 2

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2

Input

nums =
`[1,3,5,7]`

Output

`-4`

Expected

`-4`

Description | Note | Editorial | Solutions | Submissions

1512. Number of Good Pairs

Solved

Easy | Topics | Companies | Hint

Given an array of integers `nums`, return *the number of* ***good pairs***.

A pair `(i, j)` is called *good* if `nums[i] == nums[j]` and `i` < `j`.

Example 1:

```
Input: nums = [1,2,3,1,1,3]
Output: 4
Explanation: There are 4 good pairs (0,3), (0,4), (3,4), (2,5) 0-indexed.
```

Example 2:

```
Input: nums = [1,1,1,1]
Output: 6
Explanation: Each pair in the array are good.
```

Example 3:

```
Input: nums = [1,2,3]
Output: 0
```

Constraints:

- `1 <= nums.length <= 100`
- `1 <= nums[i] <= 100`

5.9K | 142 | 21 Online

Code

Java | Auto

```java
class Solution {
    public int numIdenticalPairs(int[] nums) {
        int count=0;
        for(int i=0;i<nums.length;i++){
            for(int j=i+1;j<nums.length;j++){
                if (nums[i]==nums[j]){
                    count++;
                }
            }
        }
      return count;
    }
```

Saved

Ln 1, Col 1

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

nums =

[1,2,3,1,1,3]

Output

4

Expected

4

Description | Note | Editorial | Solutions | Submissions

3467. Transform Array by Parity

Solved

Easy | Topics | Companies | Hint

You are given an integer array `nums`. Transform `nums` by performing the following operations in the **exact** order specified:

1. Replace each even number with 0.

2. Replace each odd numbers with 1.

3. Sort the modified array in **non-decreasing** order.

Return the resulting array after performing these operations.

Example 1:

> **Input:** `nums = [4,3,2,1]`
>
> **Output:** `[0,0,1,1]`
>
> **Explanation:**
>
> - Replace the even numbers (4 and 2) with 0 and the odd numbers (3 and 1) with 1. Now, `nums = [0, 1, 0, 1]`.
> - After sorting `nums` in non-descending order, `nums = [0, 0, 1, 1]`.

Example 2:

> **Input:** `nums = [1,5,1,4,2]`
>
> **Output:** `[0,0,1,1,1]`
>
> **Explanation:**

115 | 32 | 2 Online

Code

Java | Auto

```
10
11          for (int i = 0; i < even; i++) {
12              nums[i] = 0;
13          }
14
15          for (int i = even; i < nums.length; i++) {
16              nums[i] = 1;
17          }
18
19          return nums;
20      }
21  }
```

Saved | Ln 21, Col 2

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2

Input

nums =
`[4,3,2,1]`

Output

`[0,0,1,1]`

Expected

`[0,0,1,1]`

Description | Note | Editorial | Solutions | Submissions

3289. The Two Sneaky Numbers of Digitville

Solved

Easy | Topics | Companies | Hint

In the town of Digitville, there was a list of numbers called `nums` containing integers from `0` to `n - 1`. Each number was supposed to appear **exactly once** in the list, however, **two** mischievous numbers sneaked in an *additional time*, making the list longer than usual.

As the town detective, your task is to find these two sneaky numbers. Return an array of size **two** containing the two numbers (in *any order*), so peace can return to Digitville.

Example 1:

> **Input:** `nums = [0,1,1,0]`
>
> **Output:** `[0,1]`
>
> **Explanation:**
>
> The numbers 0 and 1 each appear twice in the array.

Example 2:

> **Input:** `nums = [0,3,2,1,3,2]`
>
> **Output:** `[2,3]`
>
> **Explanation:**
>
> The numbers 2 and 3 each appear twice in the array.

537 | 161 | 3 Online

Code

Java | Auto

```
6
7           for (int num : nums) {
8               count[num]++;
9               if (count[num] == 2) {
10                  ans[k++] = num;
11              }
12          }
13
14          return ans;
15      }
16  }
```

Saved | Ln 16, Col 2

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

nums =
`[0,1,1,0]`

Output

`[1,0]`

Expected

`[0,1]`

Description | Note | Editorial | Solutions | Submissions

2574. Left and Right Sum Differences

Solved

Easy | Topics | Companies | Hint

You are given a **0-indexed** integer array `nums` of size `n`.

Define two arrays `leftSum` and `rightSum` where:

- `leftSum[i]` is the sum of elements to the left of the index `i` in the array `nums`. If there is no such element, `leftSum[i] = 0`.
- `rightSum[i]` is the sum of elements to the right of the index `i` in the array `nums`. If there is no such element, `rightSum[i] = 0`.

Return an integer array `answer` of size `n` where `answer[i] = |leftSum[i] - rightSum[i]|`.

Example 1:

```
Input: nums = [10,4,8,3]
Output: [15,1,11,22]
Explanation: The array leftSum is [0,10,14,22] and the array rightSum is [15,11,3,0].
The array answer is [|0 - 15|,|10 - 11|,|14 - 3|,|22 - 0|] = [15,1,11,22].
```

Example 2:

```
Input: nums = [1]
Output: [0]
Explanation: The array leftSum is [0] and the array rightSum is [0].
The array answer is [|0 - 0|] = [0].
```

1.5K | 153 | 25 Online

Code

Java | Auto

```java
        for (int num : nums) {
            total += num;
        }

        int leftSum = 0;

        for (int i = 0; i < n; i++) {
            total -= nums[i];               // right sum
            ans[i] = Math.abs(leftSum - total);
            leftSum += nums[i];
        }

```

Saved

Ln 21, Col 2

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2

Input

nums =
[10,4,8,3]

Output

[15,1,11,22]

Expected

[15,1,11,22]

Description | Note | Editorial | Solutions | Submissions

2574. Left and Right Sum Differences

Solved

Easy | Topics | Companies | Hint

You are given a **0-indexed** integer array `nums` of size `n`.

Define two arrays `leftSum` and `rightSum` where:

- `leftSum[i]` is the sum of elements to the left of the index `i` in the array `nums`. If there is no such element, `leftSum[i] = 0`.
- `rightSum[i]` is the sum of elements to the right of the index `i` in the array `nums`. If there is no such element, `rightSum[i] = 0`.

Return an integer array `answer` of size `n` where `answer[i] = |leftSum[i] - rightSum[i]|`.

Example 1:

```
Input: nums = [10,4,8,3]
Output: [15,1,11,22]
Explanation: The array leftSum is [0,10,14,22] and the array rightSum is [15,11,3,0].
The array answer is [|0 - 15|,|10 - 11|,|14 - 3|,|22 - 0|] = [15,1,11,22].
```

Example 2:

```
Input: nums = [1]
Output: [0]
Explanation: The array leftSum is [0] and the array rightSum is [0].
The array answer is [|0 - 0|] = [0].
```

1.5K | 153 | 19 Online

Code

Java | Auto

```java
class Solution {
    public int[] leftRightDifference(int[] nums) {
        int n = nums.length;
        int[] ans = new int[n];

        int total = 0;
        for (int num : nums) {
            total += num;
        }

        int leftSum = 0;

        for (int i = 0; i < n; i++) {
            total -= nums[i];               // right sum
            ans[i] = Math.abs(leftSum - total);
            leftSum += nums[i];
        }
```

Saved

Ln 1, Col 1

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2

Input

nums =

[10,4,8,3]

Output

[15,1,11,22]

Description | Note | Editorial | Solutions | Submissions

3300. Minimum Element After Replacement With Digit Sum

Solved

Easy | Topics | Companies | Hint

You are given an integer array `nums`.

You replace each element in `nums` with the **sum** of its digits.

Return the **minimum** element in `nums` after all replacements.

Example 1:

> **Input:** `nums = [10,12,13,14]`
>
> **Output:** 1
>
> **Explanation:**
>
> `nums` becomes `[1, 3, 4, 5]` after all replacements, with minimum element 1.

Example 2:

> **Input:** `nums = [1,2,3,4]`
>
> **Output:** 1
>
> **Explanation:**
>
> `nums` becomes `[1, 2, 3, 4]` after all replacements, with minimum element 1.

279 | 91 | 161 Online

Code

Java | Auto

```
10                  num /= 10;
11              }
12
13              if (sum < min) {
14                  min = sum;
15              }
16          }
17
18          return min;
19      }
20  }
```

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

nums =
[10,12,13,14]

Output

1

Expected

1

Contribute a testcase

Description | Note | Editorial | Solutions | Submissions

1672. Richest Customer Wealth

Solved

Easy | Topics | Companies | Hint

You are given an `m x n` integer grid `accounts` where `accounts[i][j]` is the amount of money the `i^th` customer has in the `j^th` bank. Return *the* ***wealth*** *that the richest customer has.*

A customer's **wealth** is the amount of money they have in all their bank accounts. The richest customer is the customer that has the maximum **wealth**.

Example 1:

```
Input: accounts = [[1,2,3],[3,2,1]]
Output: 6
Explanation:
1st customer has wealth = 1 + 2 + 3 = 6
2nd customer has wealth = 3 + 2 + 1 = 6
Both customers are considered the richest with a wealth of 6 each, so return 6.
```

Example 2:

```
Input: accounts = [[1,5],[7,3],[3,5]]
Output: 10
Explanation:
1st customer has wealth = 6
2nd customer has wealth = 10
3rd customer has wealth = 8
The 2nd customer is the richest with a wealth of 10.
```

Example 3:

4.9K | 104 | 27 Online

Code

Java | Auto

```java
class Solution {
    public int maximumWealth(int[][] accounts) {
        int max = 0;

        for (int i = 0; i < accounts.length; i++) {
            int sum = 0;

            for (int j = 0; j < accounts[i].length; j++) {
                sum += accounts[i][j];
            }
```

Saved

Ln 19, Col 2

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

accounts =

[[1,2,3],[3,2,1]]

Output

6

Expected

6

Contribute a testcase

Description | Note | Editorial | Solutions | Submissions

1470. Shuffle the Array

Solved

Easy | Topics | Companies | Hint

Given the array `nums` consisting of `2n` elements in the form `[x1,x2,...,xn,y1,y2,...,yn]`.

Return the array in the form `[x1,y1,x2,y2,...,xn,yn]`.

Example 1:

```
Input: nums = [2,5,1,3,4,7], n = 3
Output: [2,3,5,4,1,7]
Explanation: Since x1=2, x2=5, x3=1, y1=3, y2=4, y3=7 then the answer is [2,3,5,4,1,7].
```

Example 2:

```
Input: nums = [1,2,3,4,4,3,2,1], n = 4
Output: [1,4,2,3,3,2,4,1]
```

Example 3:

```
Input: nums = [1,1,2,2], n = 2
Output: [1,2,1,2]
```

Constraints:

- `1 <= n <= 500`
- `nums.length == 2n`

6.5K | 164 | 37 Online

Code

Java | Auto

```java
class Solution {
    public int[] shuffle(int[] nums, int n) {
        int[] ans = new int[2 * n];
        int j = 0;

        for (int i = 0; i < n; i++) {
            ans[j++] = nums[i];
            ans[j++] = nums[i + n];
        }

        return ans;
    }
}
```

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

nums =
[2,5,1,3,4,7]

n =
3

Output

[2,3,5,4,1,7]

Description | Note | Editorial | Solutions | Submissions

1684. Count the Number of Consistent Strings

Solved

Easy | Topics | Companies | Hint

You are given a string `allowed` consisting of **distinct** characters and an array of strings `words`. A string is **consistent** if all characters in the string appear in the string `allowed`.

Return *the number of **consistent** strings in the array* `words`.

Example 1:

```
Input: allowed = "ab", words =
["ad","bd","aaab","baa","badab"]
Output: 2
Explanation: Strings "aaab" and "baa" are consistent since
they only contain characters 'a' and 'b'.
```

Example 2:

```
Input: allowed = "abc", words =
["a","b","c","ab","ac","bc","abc"]
Output: 7
Explanation: All strings are consistent.
```

Example 3:

```
Input: allowed = "cad", words =
["cc","acd","b","ba","bac","bad","ac","d"]
Output: 4
Explanation: Strings "cc", "acd", "ac", and "d" are
consistent.
```

2.3K | 123 | 6 Online

Code

Java | Auto

```java
class Solution {
    public int countConsistentStrings(String allowed, String[] words) {
        int count = 0;

        for (String word : words) {
            boolean ok = true;

            for (char c : word.toCharArray()) {
                if (allowed.indexOf(c) == -1) {
                    ok = false;
                    break;
                }
```

Saved

Ln 22, Col 2

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

allowed =

"ab"

words =

["ad","bd","aaab","baa","badab"]

Output

2

Description | Note | Editorial | Solutions | Submissions

1431. Kids With the Greatest Number of Candies

Solved

Easy | Topics | Companies | Hint

There are `n` kids with candies. You are given an integer array `candies`, where each `candies[i]` represents the number of candies the `i^th` kid has, and an integer `extraCandies`, denoting the number of extra candies that you have.

Return *a boolean array* `result` *of length* `n`*, where* `result[i]` *is* `true` *if, after giving the* `i^th` *kid all the* `extraCandies`*, they will have the* ***greatest*** *number of candies among all the kids, or* `false` *otherwise*.

Note that **multiple** kids can have the **greatest** number of candies.

Example 1:

```
Input: candies = [2,3,5,1,3], extraCandies = 3
Output: [true,true,true,false,true]
Explanation: If you give all extraCandies to:
- Kid 1, they will have 2 + 3 = 5 candies, which is the greatest among the kids.
- Kid 2, they will have 3 + 3 = 6 candies, which is the greatest among the kids.
- Kid 3, they will have 5 + 3 = 8 candies, which is the greatest among the kids.
- Kid 4, they will have 1 + 3 = 4 candies, which is not the greatest among the kids.
- Kid 5, they will have 3 + 3 = 6 candies, which is the greatest among the kids.
```

Example 2:

5K | 248 | 37 Online

Code

Java | Auto

```java
class Solution {
    public List<Boolean> kidsWithCandies(int[] candies, int extraCandies) {
        List<Boolean> ans = new ArrayList<>();

        int max = candies[0];
        for (int c : candies) {
            if (c > max) {
                max = c;
            }
        }

        for (int c : candies) {
```

Saved

Ln 7, Col 27

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

candies =

[2,3,5,1,3]

extraCandies =

3

Output

[true,true,true,false,true]